universität innsbruck

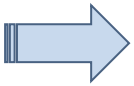
Vorschau

Einführung in die Programmierung

Michael Tschuggnall
Universität Innsbruck

Überblick

- Einführung
- Vorschau
- Variablen und Datentypen
- Operatoren und Ausdrücke
- Kontrollstrukturen
- Funktionen
- Arrays
- Zeiger
- Strukturen, Unions, Bitfelder
- Speicherklassen
- Dynamische Speicherverwaltung
- Präprozessor
- Modulare Programmierung
- Ein- und Ausgabe
- gcc und make



- Variablen und Datentypen
- Operatoren und Ausdrücke
- Kontrollstrukturen
- Zusätzliche Übungsaufgaben

Disclaimer

- Dieser Foliensatz ist eine Kurzfassung von Themen, die in späteren Vorlesungen ausführlich behandelt werden.
- Das Ziel dieser Vorlesungseinheit ist es, einen kurzen Überblick über die kommenden Themen zu geben und rasch die Fähigkeit zu erlangen, komplexeres Verhalten zu programmieren.
- Verwenden Sie daher für die Klausurvorbereitung die Unterlagen der späteren Vorlesungseinheiten.

Motivation

- Bisher:
  - Programm gibt nur einen bestimmten Text aus.
  - Text steht beim Kompilieren fest.
- Ziel:
  - Probleme lösen, etwa
    - Alle geraden Zahlen ausgeben
    - Fibonacci-Folge berechnen
    - …
  - Dazu müssen wir:
    - Werte speichern
    - Rechnen
    - Je nach Ergebnis unterschiedlichen Code ausführen
    - Teile des Codes wiederholt ausführen

Variablen und Datentypen

Variablen (1)

- Variable:
  - Ein Programm verarbeitet **Daten**, die in sogenannten **Variablen** abgelegt werden.
  - Ein Programm legt die **Ergebnisse** wieder in solchen **Variablen** ab.
  - Eine Variable ist eine **benannte Speicherstelle**.
- Kennzeichen für eine Variable:
  - **Datentyp**
  - **Name**
  - **Wert**
  - Adresse
  - Gültigkeitszeitraum
  - Sichtbarkeitsbereich

Variablen (2)

- Datentyp:
  - Variablen haben einen Datentyp.
  - Der Datentyp legt fest:
    - **Wertebereich**
    - **Repräsentation im Speicher** (wie viele Bytes werden belegt)
    - **Operationen**, die auf den Datentyp angewandt werden dürfen
- Name (Bezeichner):
  - Durch den Bezeichner kann auf die benannte Speicherstelle zugegriffen werden.
  - Ist eine Folge aus Buchstaben, Ziffern und Unterstrichen.
  - Der Bezeichner muss aber mit einem Buchstaben oder Unterstrich beginnen.
- Wert:
  - Der Wert einer Variable kann sich ändern.
  - Der Variable muss (meistens) durch das Programm am Anfang explizit ein Wert zugewiesen werden (**Initialisierung**).
    - Ansonsten hat die Variable einen zufälligen Wert! (Ausnahmen werden später besprochen)

Variablen (3)

- Variablen müssen vor der Verwendung deklariert werden.
  - Die Deklaration legt den Datentyp und den Bezeichner fest.
- Beispiele von Deklarationen:

```c
#include <stdlib.h>

int main(void) {
    int height;
    int limit;
    char letter;
    double area;
    // ...
    return EXIT_SUCCESS;
}
```

Ausgewählte Datentypen

- `char`
  - Dient zur Aufnahme von Zeichen (oder Ganzzahlen).
- `int`
  - Wird für ganzzahlige Werte mit Vorzeichen verwendet.
- `float`
  - Wird für Gleitkommazahlen mit einfacher Genauigkeit verwendet.
- `double`
  - Wird für Gleitkommazahlen mit doppelter Genauigkeit verwendet.

Operatoren und Ausdrücke

Operatoren (1)

- Zuweisungsoperator
  - = (Zuweisungsoperator)

```
1 int a;
2 a = 1;
3 int b = 2;
4 int c = b;
```

- Arithmetische Operatoren
  - * (Multiplikation)
  - / (Division)
  - + (positives Vorzeichen, Addition)
  - - (negatives Vorzeichen, Subtraktion)
  - % (Rest)
    - Rest bei der Division von Ganzzahlen
      - $11 : 2 = 5 \quad 1\ Rest$
      - $11\ \%\ 2 = 1$

Operatoren (2)

- Vergleichsoperatoren

Operator	Beispiel	Beschreibung
==	x == y	ist der Wert von x gleich dem Wert von y
!=	x != y	ist der Wert von x ungleich dem Wert von y
>	x > y	ist der Wert von x größer als der Wert von y
<	x < y	ist der Wert von x kleiner als der Wert von y
>=	x >= y	ist der Wert von x größer oder gleich wie der Wert von y
<=	x <= y	ist der Wert von x kleiner oder gleich wie der Wert von y

- Das Ergebnis eines Vergleichs hat den Datentyp `int`.
  - In C wird der Wert 0 als **falsch** interpretiert und alle Werte ungleich 0 als **wahr**.
  - Vergleichende Operatoren ergeben den Wert 0, wenn die entsprechende Aussage falsch ist, andernfalls ist das Ergebnis 1.

Operatoren (3)

- Logische Operatoren
  - `&&` (logisches Und)
  - `||` (logisches Oder)
  - `!` (logische Negation)

x	y	!x	x && y	x || y
1*	1*	0	1	1
1*	0	0	0	1
0	1*	1	0	1
0	0	1	0	0

* oder jeder andere Wert, der ungleich 0 ist.

Beispiele Operatoren

- Gegeben seien die folgenden Deklarationen:

```
1 int x = 2;
2 int y = 8;
3 int z = 10;
```

Ausdruck	Ergebnis
`5 % 3`	2
`(y / x) - 1`	3
`(x * y) + z`	26
`(x + y) == z`	1
`(x >= y) || (x <= z)`	1
`(x != 0) && (!(y == 8))`	0

Ausgabe mit printf (1)

- Durch die Funktion `printf` kann Text formatiert auf der Standardausgabe (Konsole) ausgegeben werden.
- Allgemeine Form von `printf`:
  - `printf(`Formatstring`,` Parameterliste`)`
- Formatstring:
  - Besteht aus gewöhnlichem Text, der zwischen Anführungszeichen (") angegeben wird.
  - Im Text können Platzhalter – für die Ausgabe von Variableninhalten – eingefügt werden.
  - Die Platzhalter beginnen immer mit einem Prozentzeichen (%).
    - `%d` (Platzhalter für vorzeichenbehaftete Ganzzahlen)
  - Durch das Zeichen `\n` kann ein Zeilenvorschub eingefügt werden.

Ausgabe mit printf (2)

example_printf.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int x = 5;
    printf("Hello World!\n");
    printf("%d\n", x + 2);
    printf("x = %d\n", x);
    x = x + 1;
    printf("x = %d\n", x);
    return EXIT_SUCCESS;
}
```

Ausgabe auf der Standardausgabe:

```
Hello World!
7
x = 5
x = 6
```

Aufgabe Operatoren

- Was gibt das folgende C-Programm auf der Standardausgabe aus?

exercise_operators.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int a = 1;
    int b = 2;
    printf("1. %d\n", a + 2);
    printf("2. %d\n", (a + 1) * b);
    printf("3. %d\n", b != 2);
    printf("4. %d\n", (a < b) && (b > 0));
    printf("5. %d\n", !(b - 2 * a));
    return EXIT_SUCCESS;
}
```

Aufgabe Operatoren - Lösung

- Was gibt das folgende C-Programm auf der Standardausgabe aus?

exercise_operators.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int a = 1;
    int b = 2;
    printf("1. %d\n", a + 2);
    printf("2. %d\n", (a + 1) * b);
    printf("3. %d\n", b != 2);
    printf("4. %d\n", (a < b) && (b > 0));
    printf("5. %d\n", !(b - 2 * a));
    return EXIT_SUCCESS;
}
```

Ausgabe auf der Standardausgabe:

```
1. 3
2. 4
3. 0
4. 1
5. 1
```

Kontrollstrukturen

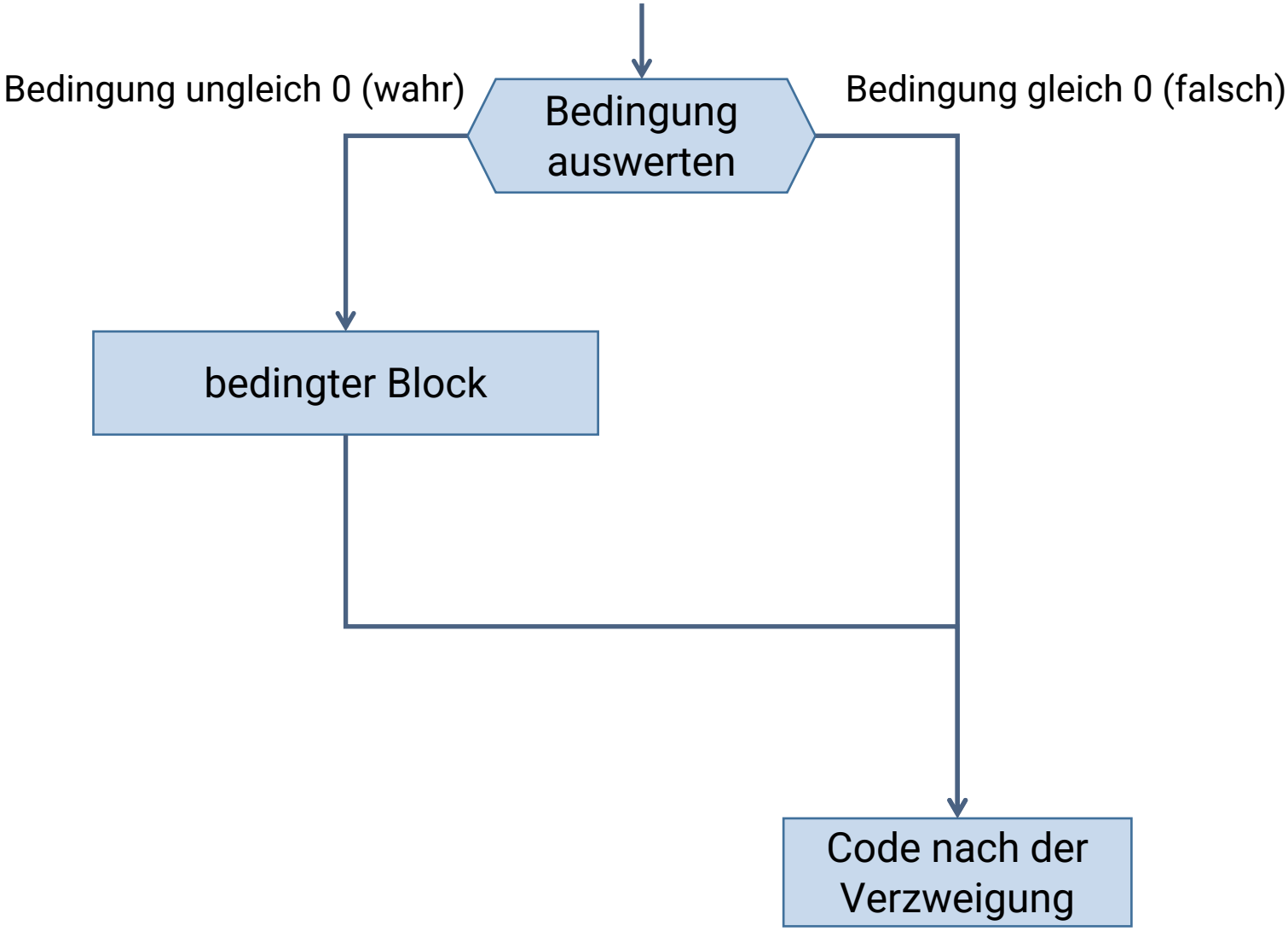
Verzweigungen (1)

- Allgemeine Form:

```
if (Bedingung) {
    Anweisungsfolge1;
}
```

- Durch Verzweigungen kann der Ablauf des Programms beeinflusst werden. Dabei kann mit logischen Bedingungen entschieden werden, an welcher Stelle das Programm fortgesetzt werden soll.
  - Der Code im `if`-Block (alles zwischen { und }) wird nur ausgeführt, wenn die Bedingung erfüllt ist (Wert ist ungleich 0).
  - Ist die Bedingung nicht erfüllt (Wert ist gleich 0), wird der Code nach dem `if`-Block ausgeführt.

Verzweigungen (2)



Verzweigungen (3)

example_if.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int a = 1;
    printf("a = %d\n", a);
    if (a % 2 == 1) {
        printf("ungerade\n");
    }
    if (a % 2 == 0) {
        printf("gerade\n");
    }
    printf("ende\n");
    return EXIT_SUCCESS;
}
```

Ausgabe auf der Standardausgabe:

```
a = 1
ungerade
ende
```

Verzweigungen (4)

example_if.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int a = 2;
    printf("a = %d\n", a);
    if (a % 2 == 1) {
        printf("ungerade\n");
    }
    if (a % 2 == 0) {
        printf("gerade\n");
    }
    printf("ende\n");
    return EXIT_SUCCESS;
}
```

Ausgabe auf der Standardausgabe:

```
a = 2
gerade
ende
```

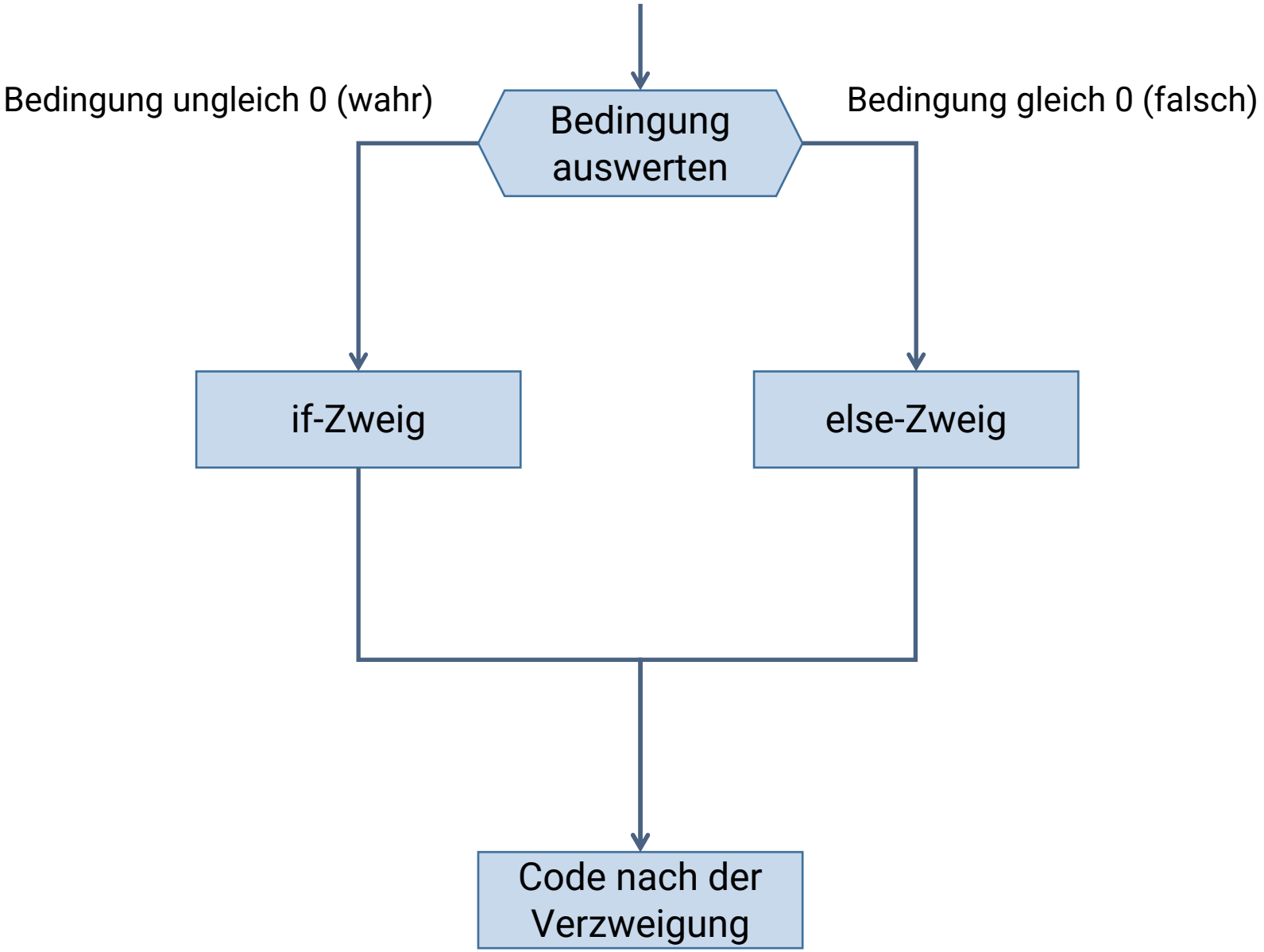
if-Anweisung mit else-Zweig (1)

- Allgemeine Form:

```
if (Bedingung) {
    Anweisungsfolge1;
} else {
    Anweisungsfolge2;
}
```

- Der `else`-Block wird immer ausgeführt, wenn die Bedingung den Wert 0 hat.

if-Anweisung mit else-Zweig (2)



if-Anweisung mit else-Zweig (3)

example_if_else.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int a = 1;

    printf("a = %d\n", a);
    if (a % 2 == 1) {
        printf("ungerade\n");
    } else {
        printf("gerade\n");
    }
    printf("ende\n");
    return EXIT_SUCCESS;
}
```

Ausgabe auf der Standardausgabe:

```
a = 1
ungerade
ende
```

Aufgabe if-Anweisungen

- Was gibt das folgende C-Programm auf der Standardausgabe aus?

exercise_multiple_if.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int a = 1;
    int b = 3;
    if (a < b) {
        printf("1");
        if (!(a - 1)) {
            printf("2");
        }
    }
    if (a + 2 == b) {
        printf("3");
    } else {
        printf("4");
    }
    printf("5");
    return EXIT_SUCCESS;
}
```

Aufgabe if-Anweisungen - Lösung

- Was gibt das folgende C-Programm auf der Standardausgabe aus?

exercise_multiple_if.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int a = 1;
    int b = 3;
    if (a < b) {
        printf("1");
        if (!(a - 1)) {
            printf("2");
        }
    }
    if (a + 2 == b) {
        printf("3");
    } else {
        printf("4");
    }
    printf("5");
    return EXIT_SUCCESS;
}
```

Ausgabe auf der Standardausgabe:

```
1235
```

Schleifen

- Problem:
  - Ausgabe der Zahlen von 1 bis 5?
  - Ausgabe der Zahlen von 1 bis 10?
  - Ausgabe der Zahlen von 1 bis 1.000.000.000?

- Lösung: Schleifen
  - Mit Schleifen können bestimmte Anweisungen mehrfach ausgeführt werden

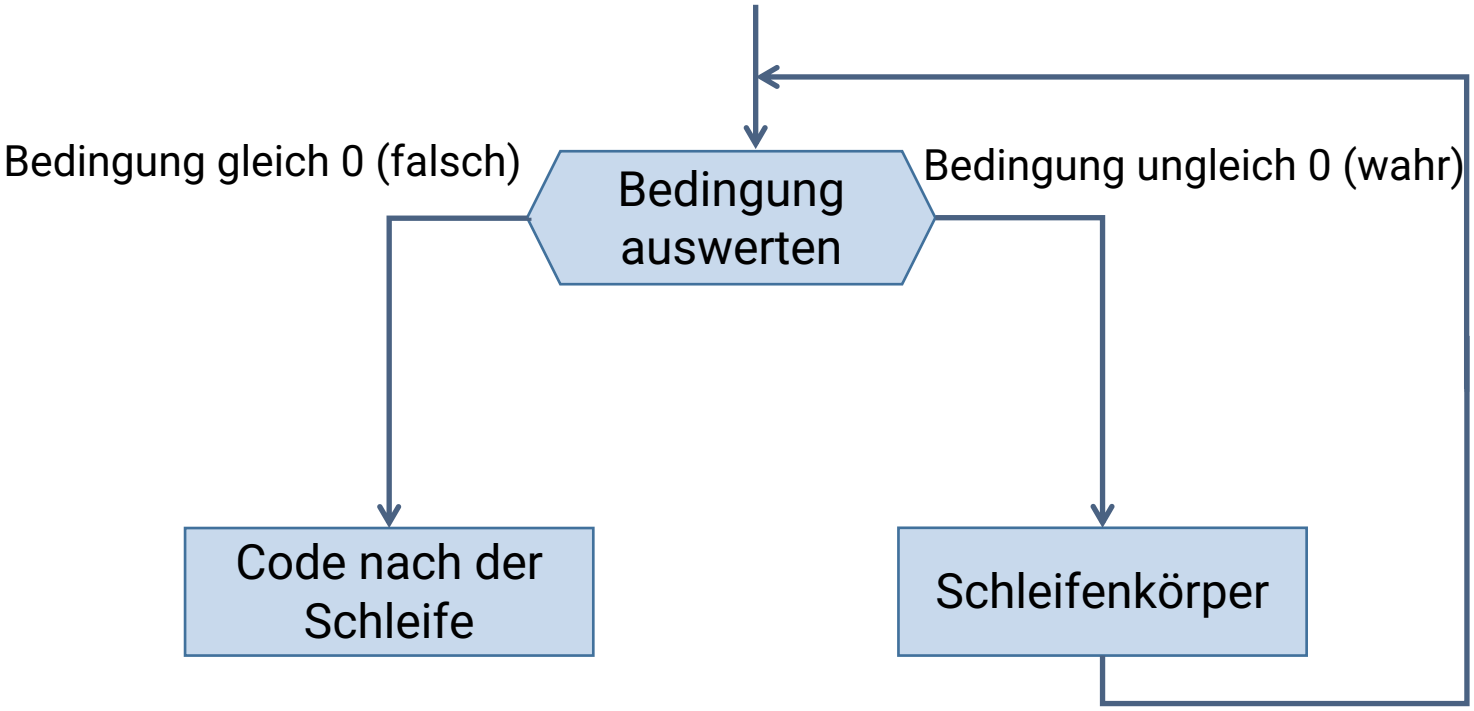
while-Schleife (1)

- Allgemeine Form

```
while (Bedingung) {
    Anweisungsfolge1;
}
```

- Die `while`-Schleife führt den Code im Schleifenkörper aus, solange die Bedingung erfüllt ist (Wert ungleich 0).
- Ablauf:
  1. Die Bedingung wird ausgewertet.
  2. Ist der Wert ungleich 0:
     - Der Schleifenkörper wird ausgeführt.
     - Nach der Ausführung des Schleifenkörpers zurück zu Schritt 1.
  3. Wenn der Wert gleich 0 ist, dann wird die nächste Anweisung nach der Schleife ausgeführt.

while-Schleife (3)



while-Schleife (4)

example_while.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int i = 1;
    while (i < 5) {
        printf("%d\n", i);
        i = i + 1;
    }
    return EXIT_SUCCESS;
}
```

Ausgabe auf der Standardausgabe:

```
1
2
3
4
```

Aufgabe while-Schleife

- Was wird durch folgende Codefragmente auf der Standardausgabe ausgegeben?

1
```c
int i = 1;
while (i < 5) {
    printf("%d\n", i);
}
```

2
```c
int i = 1;
while (i >= 0) {
    printf("%d\n", i);
    i = i - 1;
}
```

3
```c
int i = 6;
while (i < 5) {
    printf("%d\n", i);
    i = i + 1;
}
```

4
```c
int i = 1;
while (i <= 4) {
    printf("%d\n", i);
    i = i + 2;
}
```

5
```c
int i = 1;
while (i < 20) {
    printf("%d\n", i);
    i = 2 * i;
}
```

Zusätzliche Übungsaufgaben

Aufgabe if-Anweisung (1)

- Wählen Sie eine Ganzzahl n (z. B. 5).
- Verwenden Sie eine if-Anweisung, um zu testen, ob die Zahl n zwischen 1 und 10, d. h. $1 \leq n \leq 10$, ist.
  - Ist n zwischen 1 und 10 wird der Text `"valid"` ausgegeben.
  - Sonst wird der Text `"invalid"` ausgegeben.
- Testen Sie Ihr Programm mit verschiedenen Werten für n. Z. B.:
  - 1
  - 10
  - 0
  - -5
  - 8
  - 11

Aufgabe if-Anweisung (2)

- Wählen Sie eine Ganzzahl x (z. B. 5).
- Wählen Sie eine Ganzzahl y (z. B. 5).
- Deklarieren Sie eine Variable `minimum` mit dem Datentyp `int`.
- Deklarieren Sie eine Variable `maximum` mit dem Datentyp `int`.
- Verwenden Sie if-Anweisungen, um das Minimum bzw. das Maximum der Werte der Variablen x und y in der Variable `minimum` bzw. `maximum` zu speichern.
- Geben Sie die Werte der Variablen x, y, `minimum` und `maximum` aus.
- Testen Sie Ihr Programm mit verschiedenen Werten für x und y. Z. B.:
  - 5 und 5
  - -4 und 8
  - 12 und 1

Aufgabe while-Schleife (1)

- Wählen Sie eine Ganzzahl n (z. B. 5).
- Verwenden Sie eine while-Schleife, um alle Ganzzahlen von 0 bis n, d. h. $\{t \in \{0, \ldots, n\}\}$, in absteigender Reihenfolge auf der Standardausgabe auszugeben.

Aufgabe while-Schleife (2)

- Wählen Sie eine Ganzzahl n (z. B. 5).
- Verwenden Sie eine while-Schleife, um alle Ganzzahlen von 0 bis n, d. h. $\{t \in \{0, \ldots, n\}\}$, in aufsteigender Reihenfolge auf der Standardausgabe auszugeben.

Aufgabe if-Anweisung und while-Schleife

- Wählen Sie eine positive Ganzzahl n (z. B. 5).
- Ermitteln Sie die Anzahl der Teiler von n.
- Geben Sie den Wert von n und die Anzahl der Teiler von n auf der Standardausgabe aus.